

CS 312 System Administration

Ops 5: Observability and Incident Response

[Copy Markdown](#)[Open](#) ▾

The Minecraft server went down at 2 AM on a Tuesday. Nobody noticed until the 9 AM standup, when the CEO asked why his nether portal was not loading. The incident retrospective was brief: “We did not know it was down.” The CEO’s only feedback, delivered without blinking: “How do we not know things?”

Your service runs on Kubernetes, but you have no visibility into its health. When something breaks, you find out from the CEO, not from your tools. This assignment adds the operational visibility and response practices that ensure you know about problems before leadership does.

Learning Objectives

- Deploy and configure a monitoring stack (Prometheus + Grafana) on Kubernetes.
- Design actionable alerts tied to runbooks that reduce mean time to recovery.
- Execute a structured incident drill with detection, recovery, and postmortem.
- Demonstrate log access and interpretation for troubleshooting.

Constraints (AWS Academy)

- You must use AWS Academy resources only.
- Start from your Ops 4 baseline: the existing k3s deployment on EC2. Single-node k3s is expected.
- Monitoring tools run alongside Minecraft on the same cluster; choose scrape intervals, retention, and storage settings that are defensible for a one-node lab environment.
- Images remain sourced from ECR. World backup and restoreability remain in S3.
- Use the IAM instance profile pattern from earlier assignments when AWS access is required. Do not place AWS access keys in manifests, dashboards, or ad hoc scripts.

- Minimize public exposure: restrict SSH to a known source, keep only required public ports open, and do not expose Prometheus or Grafana publicly without authentication and justification.
- Document cost controls: instance size, stop schedule, how to tear down the monitoring stack, and at least one retention or storage guardrail that limits monitoring growth.

Requirements

A. Monitoring Stack and Dashboard

- Deploy Prometheus and Grafana (or instructor-approved equivalent) on your k3s cluster.
- The monitoring deployment must be declarative and version-controlled: Helm values, Kubernetes manifests, or an equivalent reproducible path.
- Collect and visualize node health metrics: CPU, memory, and disk usage.
- Collect and visualize pod or workload health metrics for Minecraft: restarts, readiness, and resource consumption.
- Collect and visualize at least one Minecraft-specific signal such as player count, JVM memory, an RCON or status-exporter signal, or service response behavior. Document what the signal means and why it is useful.
- Build a dashboard that answers the operator question: "Is my service healthy right now?" Panels must be clearly labeled and useful to another operator under time pressure.

B. Actionable Alerts

- Define at least **three** alerts.
- Each alert must be tied to a clear operator symptom, not just an unexplained raw threshold.
- Each alert must include a justified threshold or trigger condition.
- Each alert must link to a runbook section with first-response steps.
- At least one alert must reflect player-visible service impact or loss of availability, not only host saturation.

Examples: pod crash looping, sustained high memory usage approaching limits, disk usage above 80%, service probe failures.

C. Incident Drill and Log Investigation

- Pick **one** failure scenario and execute it end-to-end:
 - **Bad deploy**: push a deployment with a broken image, detect via alerts/dashboard, recover via rollback
 - **Resource exhaustion**: set artificially low resource limits, detect via metrics, recover via adjustment
 - **Service misconfiguration**: introduce a bad ConfigMap change, detect via probes/logs, recover via config fix
- Your drill documentation must name the introduced failure, the specific detection path, the recovery steps, and a brief postmortem.
- During the drill, use at least one authoritative investigation step such as `kubectl describe`, `kubectl get events`, or `kubectl logs`. When relevant, `kubectl logs --previous` is acceptable.
- Demonstrate that you can locate and interpret a specific log line or event from the drill.
- Document where logs live for this cluster today and what an operator should search first during an incident.

D. Documentation

Your PDF must be usable by another operator with no prior knowledge of your setup. Required sections:

- Updated architecture diagram showing the Minecraft workload, Prometheus, Grafana, any exporters, and the existing ECR and S3 integration points.
- On-call quickstart: what to check first when paged, where to look next, and how to tell whether the problem is user-visible.
- One runbook per alert (at least 3 runbooks).
- Link to your repository and a concise file map identifying the exact manifests, Helm values, dashboards, alert definitions, and supporting automation you used.
- Cost controls: how to tear down monitoring components, what retention or storage choices limit growth, and any scheduling considerations.

Hints

These pointers cover Minecraft-specific integration points.

- Default dashboards are not enough. Build at least one focused dashboard that puts node saturation, pod health, and your Minecraft-specific signal on one screen.
- If you need a Minecraft-specific signal, an RCON exporter, JVM metric source, or a status utility such as [mc-monitor](#) can be a defensible starting point. Whatever you choose, explain what “healthy” means for that signal.
- If you use a simpler Prometheus deployment, verify that you are actually collecting the Kubernetes metadata and workload metrics you plan to alert on. A dashboard cannot show pod restarts or readiness if nothing is scraping that layer.

You can use any Minecraft server software you like. You can choose the Linux distribution you prefer.

What You'll Submit

1. **Observability and Incident Response Documentation (PDF)**: includes your incident drill report and brief postmortem, updated architecture diagram, on-call quickstart, alert runbooks, cost controls, and a repository link with a concise file map for the submitted manifests, Helm values, dashboards, alert definitions, and supporting automation. Another operator should be able to detect, investigate, and recover a failure from this document without asking you questions.
2. **Narrated screen recording (max 3 minutes)**. Your server MOTD must include your name or student ID. **Submit timestamps alongside the video** (e.g., “Checkpoint 1: 0:00, Checkpoint 2: 0:38, ...”):
 1. Run `nmap -sV -Pn -p T:25565 <public-endpoint>` showing the service reachable with your custom MOTD, then show the Grafana dashboard with current node health, pod health, and one Minecraft-specific signal.
 2. Show your alert definitions and one linked runbook. Explain why one alert threshold or trigger condition is defensible.
 3. Use `kubectl logs` to locate a specific event relevant to your chosen drill or a recent failure. Make it clear what the log line means and where an operator would look next.

4. Execute your incident drill: introduce the failure, show detection in the dashboard, alerts, or logs, perform recovery, and confirm the service is healthy again.

Rubric

Always refer to Canvas for the most up-to-date rubric information. Canvas's rubric will be used for grading.

Observability and Incident Response (Total: 100 pts)

Criteria	Ratings
Video: Baseline observability and reachability (10)	10 pts
Video checkpoint 1: <code>`nmap -sV -Pn -p T:25565 <public-endpoint>`</code> shows the Minecraft service reachable, the MOTD contains name or student ID, and Grafana shows current node health, pod or workload health, and one Minecraft-specific signal.	Complete All four elements clearly shown: nmap reachability, MOTD with name or student ID, node or workload health panels, and a Minecraft-specific signal visible in Grafana. 5 pts Partial Three of four elements clearly shown, or one is ambiguous (for example, MOTD missing name or ID, Grafana missing one signal category, or nmap output incomplete). 0 pts Missing No credible baseline observability and reachability demonstration.
Video: Alert definition and runbook linkage (10)	10 pts
Video checkpoint 2: at least three alert definitions are shown, one alert threshold or trigger condition is explained, and the linked runbook steps for that alert are visible.	Complete All three elements clearly shown: three or more alerts visible, one threshold or trigger condition explained, and linked runbook steps shown. 5 pts Partial

Criteria**Ratings**

	Two of three elements clearly shown, or one is ambiguous (for example, fewer than three alerts visible, threshold explanation unclear, or runbook linkage not visible). 0 pts Missing No credible alert-definition and runbook-linkage demonstration.
Video: Log investigation with kubectl logs (10) Video checkpoint 3: `kubectl logs` or `kubectl logs --previous` is used to find a specific event relevant to the chosen drill or a recent failure; the log line or event is interpreted; the next investigation step or operator action is stated.	10 pts Complete All three elements clearly shown: targeted log lookup, interpretation of the event, and the next operator step stated. 5 pts Partial Two of three elements clearly shown, or one is ambiguous (for example, logs shown without interpreting the event, or no next step is stated). 0 pts Missing No credible log-investigation demonstration.
Video: Incident drill detection and recovery (10) Video checkpoint 4: failure is clearly introduced, detection is shown through a dashboard panel, alert, or log signal, recovery is executed, and the service is confirmed healthy again.	10 pts Complete All four elements clearly shown: failure introduced, detection signal shown, recovery executed, and service confirmed healthy again. 5 pts Partial Three of four elements clearly shown, or one is ambiguous (for example, failure introduction unclear, detection not visible, or recovery not confirmed). 0 pts

Criteria	Ratings
	Missing No credible incident-drill detection and recovery demonstration.
Monitoring stack deployment and integration (15) Evaluated on four elements: (1) Prometheus and Grafana or equivalent are deployed on the k3s cluster, (2) the deployment and configuration are declarative and version-controlled, (3) both cluster or node metrics and pod or workload metrics are collected successfully, (4) the access pattern is documented and public exposure is minimized or justified.	15 pts All four elements All four elements present and correctly implemented. 11 pts Three elements Three of four elements present; one meaningful gap remains. 7 pts Two elements Two of four elements present; significant missing functionality or weak operational control remains. 0 pts One or zero elements Fewer than two elements present or the monitoring stack is not operational.
Dashboard: operator health view and service-specific signal (15) Evaluated on four elements: (1) the dashboard answers "is my service healthy right now?", (2) node health panels are meaningful and labeled, (3) pod or workload health panels are meaningful and labeled, (4) at least one Minecraft-specific signal is collected, visualized, and explained clearly enough for another operator to interpret.	15 pts All four elements All four elements present and clearly useful to an operator. 11 pts Three elements Three of four elements present; one panel category or explanation is weak. 7 pts Two elements Two of four elements present; dashboard coverage is significantly incomplete. 0 pts

Criteria	Ratings
	One or zero elements Fewer than two elements present or the dashboard is not usable for operator decisions.
Alerts and runbooks quality (15) Evaluated on four elements: (1) at least three alerts are defined, (2) each alert is tied to a clear symptom rather than an unexplained raw threshold, (3) thresholds or trigger conditions are justified, (4) each alert links to a runbook with first-response steps and at least one alert reflects player-visible service impact or loss of availability.	15 pts All four elements All four elements present and defensible. 11 pts Three elements Three of four elements present; one area is weaker or under-justified. 7 pts Two elements Two of four elements present; significant gaps remain in alert design or runbook linkage. 0 pts One or zero elements Fewer than two elements present or the alert set is not operationally meaningful.
Incident investigation and operator documentation (15) Evaluated on five elements: (1) the incident drill report names the introduced failure, detection path, recovery steps, and brief postmortem, (2) kubectl log access and search process are documented clearly, (3) the architecture diagram shows Minecraft, Prometheus, Grafana, exporters, and the existing ECR and S3 integration, (4) the on-call quickstart is actionable for another operator, (5) the repository link or	15 pts All five items All five items present, accurate, and usable by another operator without clarifying questions. 11 pts Four items Four of five items present; one area has a minor but meaningful gap. 7 pts Three items Three of five items present; multiple sections are thin or incomplete. 0 pts

Criteria	Ratings
file map and cost controls are complete and usable.	Two or fewer items Fewer than three items present or the documentation is not operator-usable.

Extra Credit (up to +10)

- **Log Aggregation (+5):** Deploy Loki, Promtail, or equivalent on the cluster. Show searchable, centralized logs in Grafana or a dedicated UI. Document the retention policy.
- **SLO and Alert Refinement (+5):** Define a measurable SLO for your Minecraft service (for example, availability or responsiveness), add a corresponding dashboard view plus alert or recording rule, and document how you would measure and report on it over a week.

Extra credit must stay within this assignment's observability scope. Do not replace the assignment with a managed monitoring product or a full platform redesign.